DEPARTMENT OF INFORMATION TECHNOLOGY

Design and Development of a Force-Modulating Parallel-Jaw Gripper: Software, Force Prediction and Calibration System

A Mini Project Report

Submitted to Walchand College of Engineering, Sangli for the partial fulfilment of Bachelor of Technology Multidisciplinary Minor in Robotics and Automation

Submitted By

Name	PRN
Mr. Ashutosh Shingan	23610070
Mr. Abhay Patil	246200002
Mr. Chaitanya Patil	246200003
Mr. Suraj Chavan	246200005

Guided By: Prof. Rohit Patil

Multidisciplinary Minor in Robotics and Automation, Walchand College of Engineering, Sangli. Academic Year 2026 - 27

Certificate

Walchand College of Engineering, Sangli (An Autonomous Institute), Department of Robotics and Automation.

This is to certify that the mini project entitled “Design and Development of a Force-Modulating Parallel-Jaw Gripper: Software, Force Prediction and Calibration System” is a work carried out by the following students under our supervision and guidance, in partial fulfilment of the requirements for the Multi-Disciplinary Minor (MDM) in Robotics and Automation, Bachelor of Technology (Under Graduate), at Walchand College of Engineering, Sangli during the academic year 2026 - 27.

Mr. Ashutosh Shingan (PRN 23610070), Mr. Abhay Patil (PRN 246200002), Mr. Chaitanya Patil (PRN 246200003), Mr. Suraj Chavan (PRN 246200005).

Prof. Rohit Patil, Guide

Place : Sangli. Date : _____ / _____ / 20_____

Declaration

We hereby declare that the mini project report entitled “Design and Development of a Force-Modulating Parallel-Jaw Gripper: Software, Force Prediction and Calibration System”, submitted to Walchand College of Engineering, Sangli, in partial fulfilment of the requirements for the Multi-Disciplinary Minor (MDM) in Robotics and Automation, is a record of original work carried out by us under the guidance of Prof. Rohit Patil.

This work has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma. All sources of information, data, figures and ideas taken from the work of others have been duly acknowledged and cited in the References.

Mr. Ashutosh Shingan (23610070), Mr. Abhay Patil (246200002), Mr. Chaitanya Patil (246200003), Mr. Suraj Chavan (246200005)

Date : _____ / _____ / 20____. Place : Sangli

Acknowledgement

We express our sincere gratitude to our guide, Prof. Rohit Patil, for the valuable guidance, constant encouragement and critical feedback that shaped this mini project from the first idea to the working software.

We are thankful to the MDM Coordinator for coordinating the Multi-Disciplinary Minor and for the timely reviews and suggestions during the project. We also thank the Head of the Department of Robotics and Automation and the Director of Walchand College of Engineering, Sangli, for providing the facilities and the academic environment needed to carry out this work.

We thank the mechanical and electronics sub-teams of the Force Modulated Gripper project, whose gripper design and fingertip force sensing this software is built for, and every member of the team for the cooperation that made the integrated system possible. Finally, we thank our families and friends, who supported us directly or indirectly during this work.

Abstract

A gripper must squeeze an object hard enough to lift it and gently enough not to damage it, and the right force differs for every object. This mini project develops GripSense AI, the software part of a force-modulating parallel-jaw gripper. The operator describes an object (mass, size, material, friction coefficient, fragility). The system first looks for a force that was already tested and confirmed for exactly that configuration in a Firebase Realtime Database; only if none exists does it ask a large language model (LLM) for an initial recommendation. The operator tests the force on the gripper, adjusts it with a slider until the grasp holds, and confirms the result, which is saved as a separate calibration record and reused automatically, with no further LLM call, the next time the same configuration is selected. Every force is checked against a friction model $N = m(g + a)/(n \cdot \mu)$ and the gripper and object limits.

The system is a FastAPI (Python) backend and a React and TypeScript web console with a 3D view of the pick-and-place cell, live telemetry charts, a calibration cockpit, history with replay, and a light and dark theme. A separate ML Lab trains a small neural network on a synthetic dataset of 900 simulated grasp trials; on unseen trials its error is 0.47 N, against 1.00 N for the friction formula alone, and its prediction is shown in the console beside the other forces. The software is covered by 279 automated tests and runs fully offline for demonstration. Tests on the physical gripper and with real Firebase telemetry remain to be done.

Keywords: Grip force prediction, Large language model, Slider calibration, Firebase Realtime Database, FastAPI, React, Neural network, 3D visualisation

Contents

1. Introduction (1.1 Background, 1.2 Motivation, 1.3 Objectives, 1.4 Problem Statement, 1.5 Scope and Limitations, 1.6 Organisation of the Report)
2. Literature Review (2.1 Review of Existing Work, 2.2 Research Gap)
3. Proposed System and Methodology (3.1 System Architecture, 3.2 Technology Stack, 3.3 Force Resolution Workflow, 3.4 Safety Model, 3.5 Database Design, 3.6 Backend API, 3.7 Web Console, 3.8 3D Work-Cell Visualisation, 3.9 ML Lab)
4. Results and Discussion (4.1 Functional Results, 4.2 ML Model Results, 4.3 Testing, 4.4 Implementation Status, 4.5 Comparison with Existing Approaches, 4.6 Discussion)
5. Conclusion and Future Scope (5.1 Conclusion, 5.2 Future Scope)

References

List of Figures

- Figure 3.1: Overview Page of the GripSense AI Console
- Figure 3.2: Object Gallery with Calibration Status
- Figure 3.3: Calibration Cockpit
- Figure 3.4: 3D Work-Cell during a Simulated Pick-and-Place
- Figure 3.5: Follow-the-Gripper Camera during the Grasp
- Figure 3.6: Live Telemetry Page
- Figure 3.7: Calibration History with Operation Replay
- Figure 3.8: ML Lab Dashboard
- Figure 4.1: ML Learning Curve and Predicted versus Actual Force
- Figure 4.2: Learned Force for Each Object
- Figure 4.3: Material Effects Learned by the Model

List of Tables

- Table I: Summary of the Reviewed Literature
- Table II: Technology Stack
- Table III: Calibration Lookup States
- Table IV: Safety Statuses
- Table V: Realtime Database Layout
- Table VI: Backend API Endpoints
- Table VII: ML Model Configuration
- Table VIII: ML Results on Unseen Trials
- Table IX: Learned Force per Object
- Table X: Automated Tests
- Table XI: Implementation Status
- Table XII: Comparison with Existing Approaches

Chapter 1: Introduction

A robot grasps by friction: the jaws must press with at least enough force that friction carries the weight and the acceleration of the object, and with no more force than the object or the gripper can bear [1]. A fixed force does not work for every object: a value that holds a steel bracket crushes a thin glass, and a value safe for the glass lets the bracket slip. The force-modulating gripper of this project can apply different forces, but somebody has to decide which force to use for which object, and that decision has to be safe, repeatable and recorded.

This report covers the software developed by the Information Technology members of the project team: the force prediction and calibration system, the database, the web console with its 3D visualisation, and the machine-learning experiment. The mechanical design of the gripper and its force sensing are reported separately by the other sub-teams.

1.1 Background

The minimum normal force per finger for a friction grasp is $N = m(g + a)/(n \cdot \mu)$, where m is the object mass, a the lift acceleration, n the number of fingers and μ the friction coefficient between pad and object [1]. In practice μ is only estimated and real objects differ from the model, so the force has to be confirmed by testing. Large language models can give a reasoned first estimate from a text description of an object [9], and learned models can predict grasp outcomes from data [5], [6], [7], but neither replaces a physical test.

A cloud database such as the Firebase Realtime Database lets the gripper publish its sensor readings and lets the console store results that every user sees at once [3]. A web console makes the system usable from any computer without installation.

1.2 Motivation

- Choosing a grip force by trial every time wastes time and risks damaging objects; a confirmed force should be found once and then reused.
- An LLM call costs time and money and its answer can vary; it should be used only when no tested value exists.
- Operators need to see clearly which force is an estimate, which is the value being tested, what the sensor measures and which value was confirmed.
- A visual, interactive console makes the gripper understandable for operators, students and reviewers.

1.3 Objectives

- To build a backend that predicts an initial grip force with an LLM and validates it against the friction model and the gripper limits.
- To provide a slider-based calibration workflow in which the operator tests, adjusts and explicitly confirms the force.
- To store predictions, calibrations, operations and telemetry in a Firebase Realtime Database and reuse confirmed forces automatically.
- To build a web console with live telemetry, a 3D view of the work-cell, history and replay.

- To train a machine-learning model that predicts the required force from object properties and compare it with the formula.
- To make the whole system testable and runnable offline for demonstration.

1.4 Problem Statement

Given an object described by its mass, dimensions, material, friction coefficient and fragility, the system must propose a safe grip force, let the operator confirm it on the gripper, save the confirmed value against that exact configuration, and reuse it automatically later, without fabricating sensor readings and without calling the LLM when a valid calibration already exists.

1.5 Scope and Limitations

The scope covers the software: backend, database design, web console, 3D visualisation and the ML experiment. The following limitations apply:

- The system has been exercised with simulated telemetry; it has not yet been run against the physical gripper or with live Firebase telemetry.
- The offline demonstration uses a labelled physics heuristic in place of the LLM, and an in-memory stand-in for Firebase.
- The ML model is trained on a synthetic dataset generated from a physics model with noise, not on real grasp trials.
- The browser cannot move the gripper: the selected force is a target that the hardware must apply.
- The gripper force limits are default values until the hardware team supplies validated ones.

1.6 Organisation of the Report

Chapter 2 reviews related work on grasp force, learned grasping, language models in robotics and the software tools used, and identifies the research gap. Chapter 3 presents the architecture, the technology stack, the force resolution workflow, the safety model, the database, the API, the console, the 3D view and the ML Lab. Chapter 4 reports the results, the tests and the implementation status and compares the work with existing approaches. Chapter 5 concludes the report and lists the future scope.

Chapter 2: Literature Review

This chapter groups the reviewed work by theme, compares the approaches and identifies what the present project adds.

2.1 Review of Existing Work

2.1.1 Grasp force and friction

Monkman et al. [1] give the design rules for mechanical grippers, including the friction-grasp condition used here to check every force. The formula needs an estimate of μ and a safety factor, and it says nothing about how a confirmed value is found or stored.

2.1.2 Learned grasping

Levine et al. [5] trained a deep network on large-scale real grasp attempts to predict grasp success, and Mahler et al. [6] trained grasp-quality networks on synthetic data labelled by analytic grasp metrics. Calandra et al. [7] combined vision and touch to decide how to regrasp. These works show that grasp behaviour can be learned from data, including synthetic data, but they need large datasets and focus on where to grasp rather than how hard to squeeze a known object.

2.1.3 Language models in robotics

Large language models can reason about objects from text [9]. Ahn et al. [8] grounded LLM suggestions in what a robot can actually do, and noted that LLM output must be checked against the real system. The same principle is applied here: the LLM gives only a starting point, which is validated and then confirmed by a physical test.

2.1.4 Software tools

FastAPI [2] provides typed Python web APIs with automatic request validation through Pydantic [11]. The Firebase Realtime Database [3] stores JSON data and pushes changes to connected clients in real time. React [4] and Three.js with React Three Fiber [10] allow interactive 3D views in the browser. Kingma and Ba [12] describe the Adam optimiser used to train the network, and Goodfellow et al. [13] the multilayer perceptron it is built on.

Table I: Summary of the Reviewed Literature

Ref.	Focus	Method or source	Relevance to this project
[1]	Grasp force	Textbook	Friction model used to check every force
[5]	Learned grasp success	Deep learning, real data	Grasp outcomes can be learned from trials
[6]	Grasp quality	Synthetic data, analytic labels	Training on synthetic data, as in the ML Lab
[7]	Regrasping	Vision and touch	Feedback from tests improves the grasp

Ref.	Focus	Method or source	Relevance to this project
[8], [9]	LLMs for robots	Language models	LLM output used only as a checked starting point
[2], [3], [4], [10], [11]	Software tools	Documentation	Backend, database, console and 3D view
[12], [13]	Neural networks	Paper, textbook	MLP and Adam used in the ML Lab

2.2 Research Gap

The friction formula gives a minimum but not a confirmed working value; learned grasping needs large datasets; and LLM estimates are not reliable on their own. What is missing for a small gripper is a simple workflow that starts from an estimate, confirms it with a physical test, records the confirmed value against the exact object configuration, and reuses it without repeating the estimate. This project builds that workflow and adds a learned model as a second, comparable estimate.

Chapter 3: Proposed System and Methodology

GripSense AI has four parts: a web console in the browser, a FastAPI backend, a Firebase Realtime Database, and a separate ML Lab service. The backend holds all secrets (the LLM key and the database service account) and performs every privileged write; the browser only reads live telemetry directly from the database.

3.1 System Architecture

- **Web console (React, TypeScript):** object selection, the calibration cockpit with the force slider, live telemetry, the 3D work-cell, history and replay, settings.
- **Backend (FastAPI, Python):** force resolution, the LLM client, the safety model, calibration sessions, operations, and the link to the ML Lab.
- **Firebase Realtime Database:** predictions, calibration records, the index of active calibrations, operation logs and the gripper telemetry node.
- **ML Lab (Python):** trains the force model and answers prediction requests from the backend on its own port (8100).
- **Gripper:** publishes its measured force, finger opening, object detection, slip and operation state to the telemetry node.

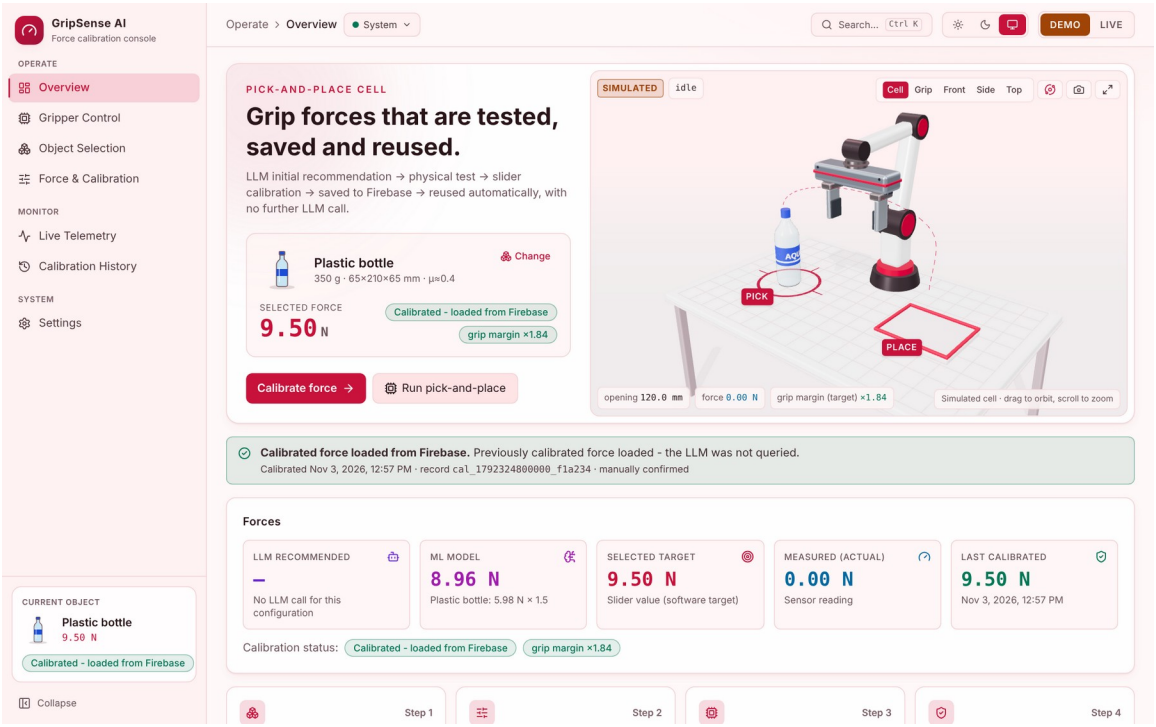


Figure 3.1: Overview Page of the GripSense AI Console

3.2 Technology Stack

Table II: Technology Stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite 8, Tailwind CSS 4	Web console
3D view	Three.js, React Three Fiber	Pick-and-place work-cell
Charts and state	Recharts, Zustand, React Router	Telemetry charts, app state, pages
Backend	Python, FastAPI, Uvicorn, Pydantic v2	API, validation, settings
LLM	Anthropic or any OpenAI-compatible API, offline heuristic	Initial force recommendation
Database	Firebase Realtime Database, Admin SDK, Web SDK	Records and live telemetry
ML Lab	Python standard library (own MLP and Adam)	Force model and dashboard
Testing	pytest, Vitest, Testing Library, Playwright	Automated and browser tests

3.3 Force Resolution Workflow

Each object configuration gets a deterministic configuration key: a SHA-256 hash of the normalised object properties and the gripper identity. Changing any property, such as the mass or the friction coefficient, gives a different key, so a calibration is reused only for exactly the configuration it was confirmed for. The workflow is:

- The operator selects or edits an object (Figure 3.2).
- The backend looks up the active calibration for the key in Firebase. If a valid one exists, its force is loaded and the LLM is not called.
- Otherwise the LLM is asked once for an initial recommendation, which is checked by the safety model and stored as a prediction.
- The operator tests the force on the gripper, adjusts it with the slider and records the result of each test (held, slipping, damage risk, failed). After a slip the assistant suggests a higher force, but never applies it automatically.
- When the grasp holds, the operator confirms it explicitly: held and lifted, no slip, no damage. Reaching the target force alone is not accepted as proof.
- The confirmed force is saved as a new calibration record, the original LLM value is kept unchanged, and the active index is updated in a transaction.

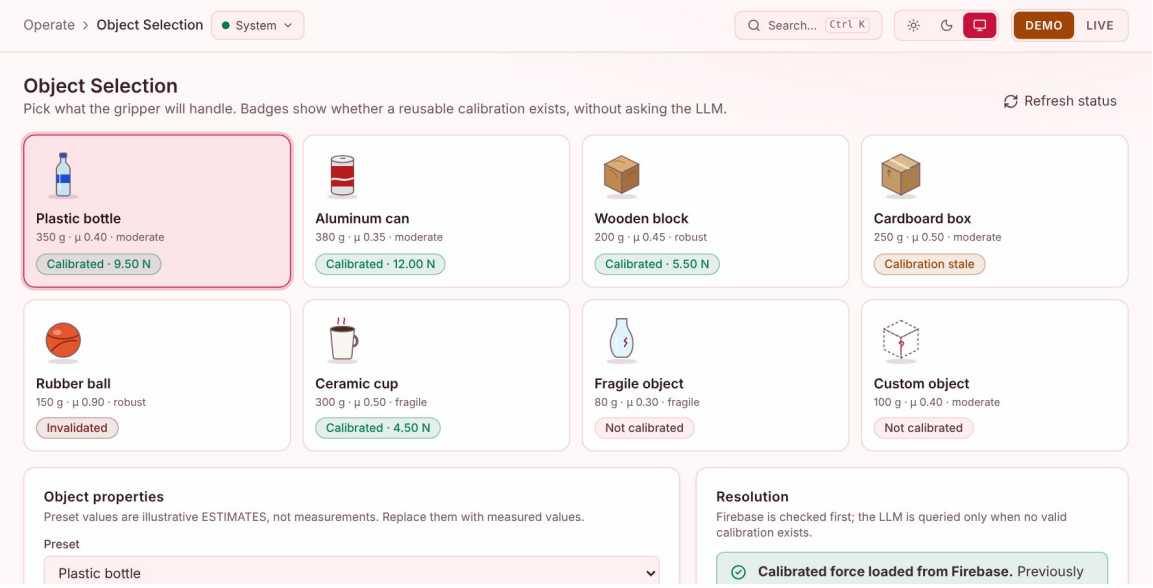


Figure 3.2: Object Gallery with Calibration Status

Table III: Calibration Lookup States

State	Meaning	Action
found_valid	A confirmed calibration exists for the key	Reuse it; no LLM call
not_found	No calibration for this configuration	Ask the LLM once
stale	Older than the allowed age	Recalibrate
invalidated	Withdrawn by an operator	Recalibrate
safety_failed	Fails the current safety check	Not applied
inconsistent	Index and record disagree	Not applied; reported

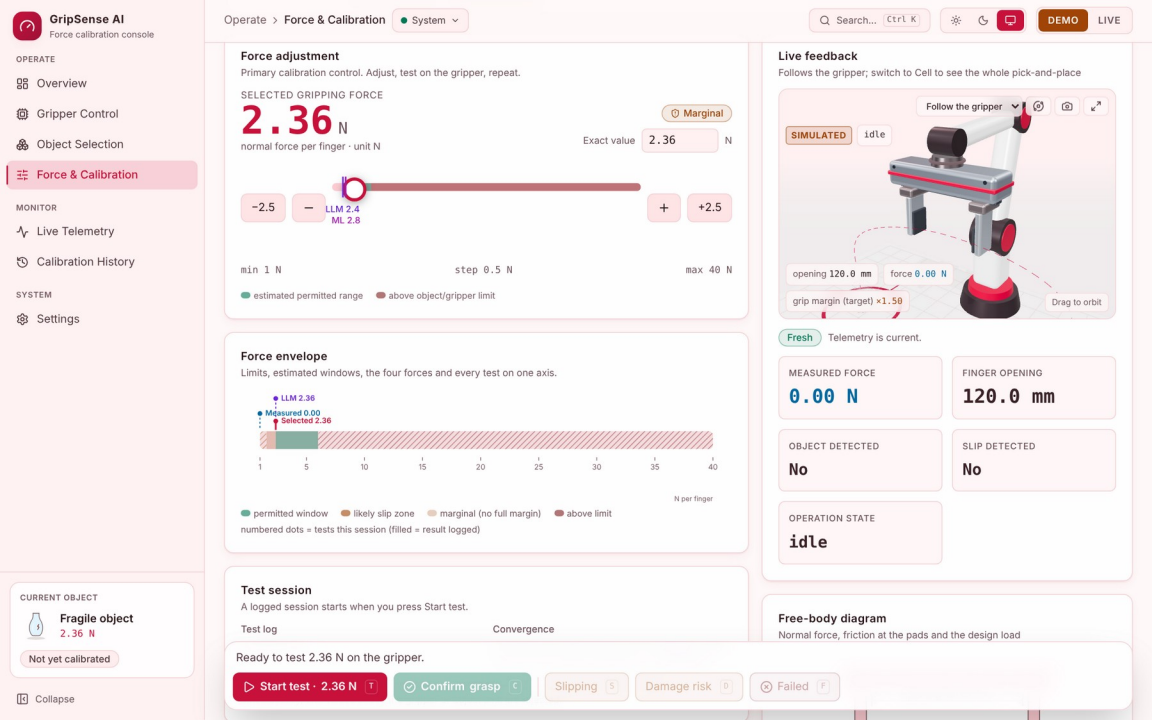


Figure 3.3: Calibration Cockpit

3.4 Safety Model

Every force shown, tested or saved is checked with the friction model N_req = m(g + a)/(n·μ), a holding safety factor (1.5), the gripper force range and the object's maximum contact force. The grip margin, N/N_req, is shown in the console and in a free-body diagram. The same rules run in the backend and in the browser and are verified against one shared set of test cases.

Table IV: Safety Statuses

Status	Condition	Effect
ok	Between the recommended force and the limits	Can be tested and saved
marginal	Above N_req but below N_req × safety factor	Can be tested; warning
below_required	Below N_req	Likely to slip; saving needs an override
below_minimum	Below the gripper minimum	Blocked
exceeds_limit	Above the gripper or object limit	Blocked
no_safe_range	No force satisfies all limits	Blocked

3.5 Database Design

Table V: Realtime Database Layout

Path	Content	Written by
grip_predictions/{id}	LLM recommendation with inputs and safety result	Backend
grip_calibrations/{id}	Every calibration session, tests and outcome (audit)	Backend
active_calibrations/{key}	The calibration reused for each configuration key	Backend (transaction)
operations/{id}	Logged pick-and-place runs with their traces	Backend
gripper_telemetry	Measured force, opening, detection, slip, state, timestamp	Gripper
demo/…	The same layout for demonstration data, kept separate	Backend

A sample export with 8 predictions, 11 calibration records in every status, 7 active entries and 10 operations was generated by running the real workflow. A validator checks any export against the application, and a seeding tool loads it.

3.6 Backend API

Table VI: Backend API Endpoints

Endpoint	Purpose
GET /api/health	Database, LLM and gripper configuration status
GET /api/objects	Object presets
POST /api/predict-grip	Resolve a configuration: saved calibration first, otherwise the LLM
POST /api/calibrations/lookup	Calibration status of many objects at once (never calls the LLM)
POST /api/calibrations	Start a calibration session
POST /api/calibrations/{id}/tests	Record a test and its result
POST /api/calibrations/{id}/confirm	Confirm and save the calibrated force
POST /api/calibrations/{id}/invalidate	Withdraw a calibration

Endpoint	Purpose
GET /api/calibrations, /api/calibration-history/{key}	History
GET, POST /api/operations	Operation logs
GET /api/telemetry/latest	Latest telemetry with freshness
POST /api/ml-prediction	The ML Lab's force for an object (display only)

3.7 Web Console

The console has seven pages: Overview, Gripper Control, Object Selection, Force and Calibration, Live Telemetry, Calibration History and Settings. Five forces are always shown separately and in distinct colours: the LLM recommendation, the ML model, the selected target, the measured force and the last calibrated force. Missing readings are shown as "unavailable", never as zero. The console has a command palette (Ctrl K), keyboard shortcuts for testing (T, S, C), a collapsible sidebar, a phone layout and light and dark themes. Telemetry is marked stale when it stops updating (Figure 3.6), and the history keeps every record for audit, with operation replay (Figure 3.7).

3.8 3D Work-Cell Visualisation

The 3D view (Three.js) shows a table with pick and place stations, a schematic arm and a detailed two-finger parallel gripper. The finger opening, the force label and the slip alert come from the telemetry; the arm follows the reported operation state through a closed-form inverse-kinematics solution. It offers five camera views, including one that follows the gripper (Figure 3.5), and renders only when something moves so the console stays responsive.

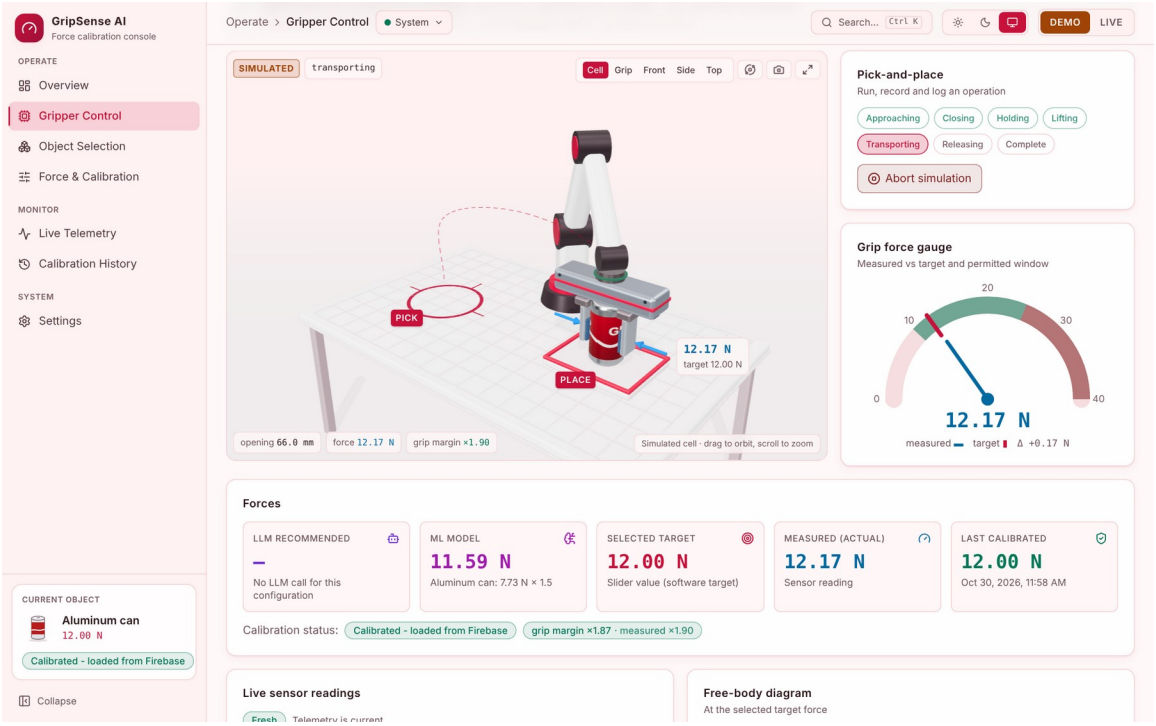


Figure 3.4: 3D Work-Cell during a Simulated Pick-and-Place

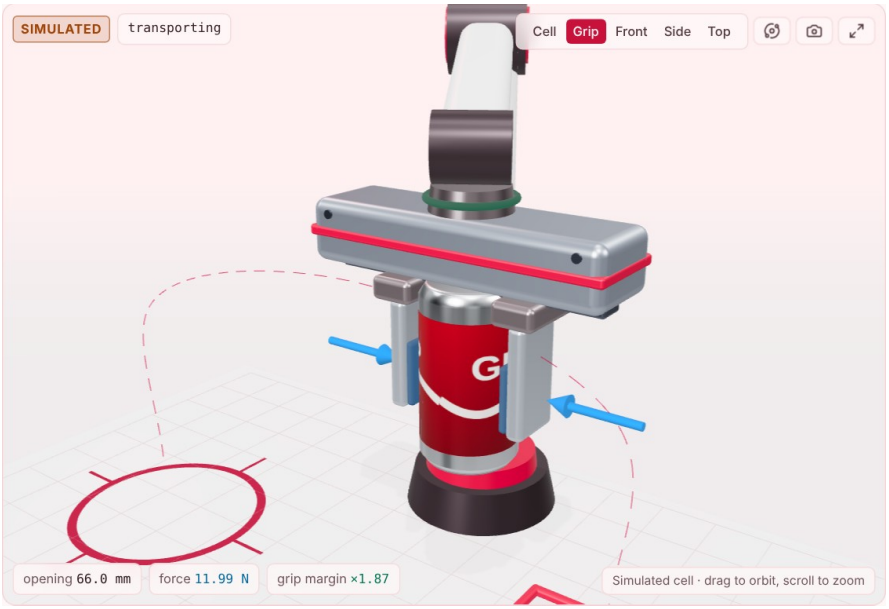


Figure 3.5: Follow-the-Gripper Camera during the Grasp

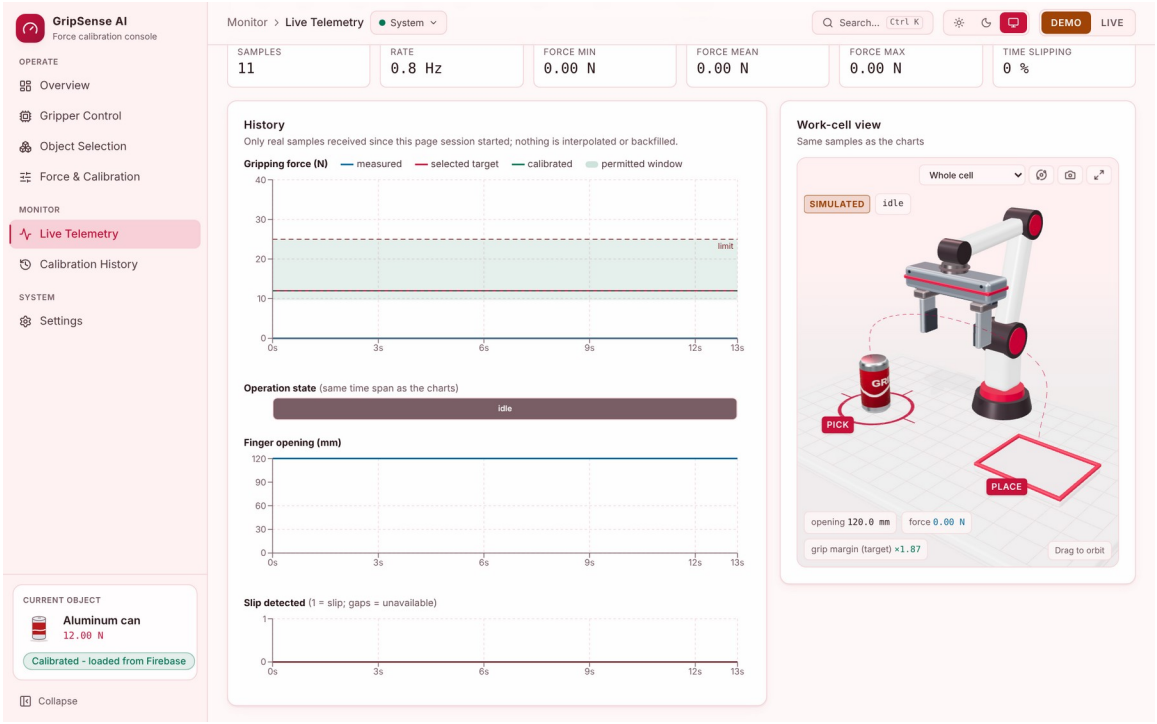


Figure 3.6: Live Telemetry Page

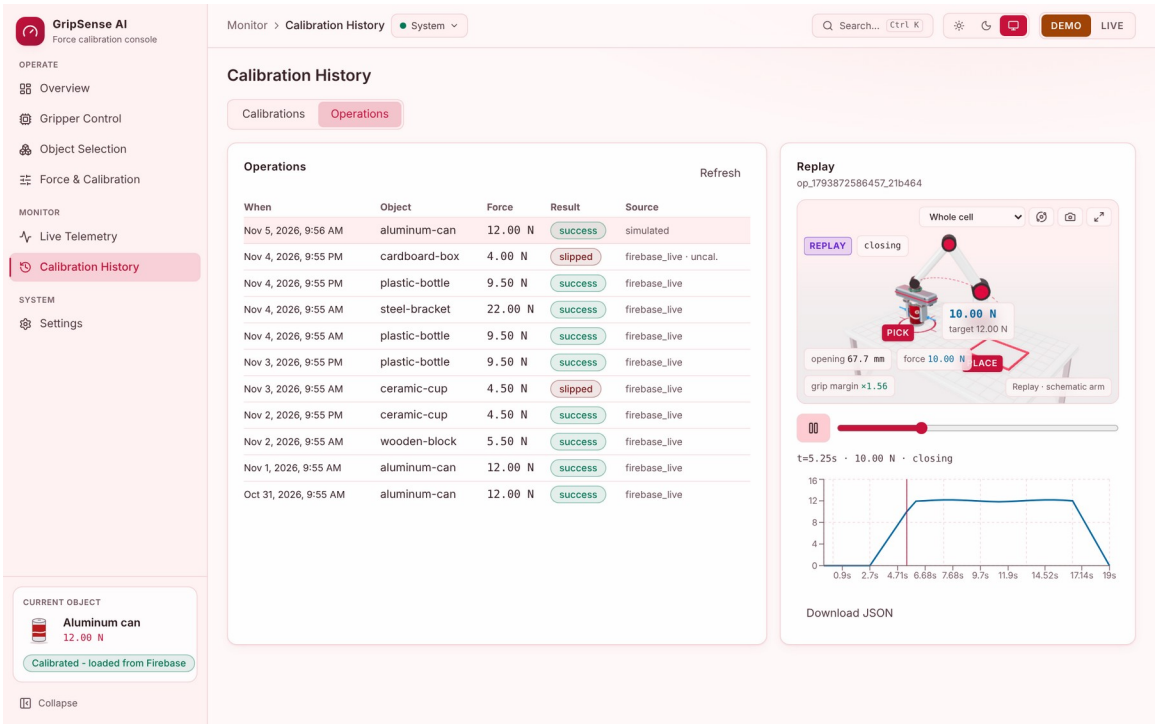


Figure 3.7: Calibration History with Operation Replay

3.9 ML Lab

The ML Lab is a separate service that trains a neural network to predict the minimum holding force. Because no real trials were available, its dataset of 900 grasp trials is synthetic: each row simulates one calibration, generated from the friction model plus effects the formula does not include (a material-specific error in the estimated μ, extra force for narrow objects, measurement noise and the slider step). The network must learn these effects from the rows alone. 720 trials are used for training and 180 are held out for validation. The dashboard shows the training live, and GripSense shows the trained model's force for the selected object.

Table VII: ML Model Configuration

Item	Value
Inputs (17)	log mass, log μ, width, height, fragility (3, one-hot), material (10, one-hot)
Target	log of the minimum holding force
Network	Multilayer perceptron, one hidden layer of 16 tanh units
Training	Adam, mini-batch 32, learning rate 0.01 halving every 40 epochs, 150 epochs
Data	900 synthetic trials; 720 training, 180 validation
Output in GripSense	Minimum force × 1.5 holding margin

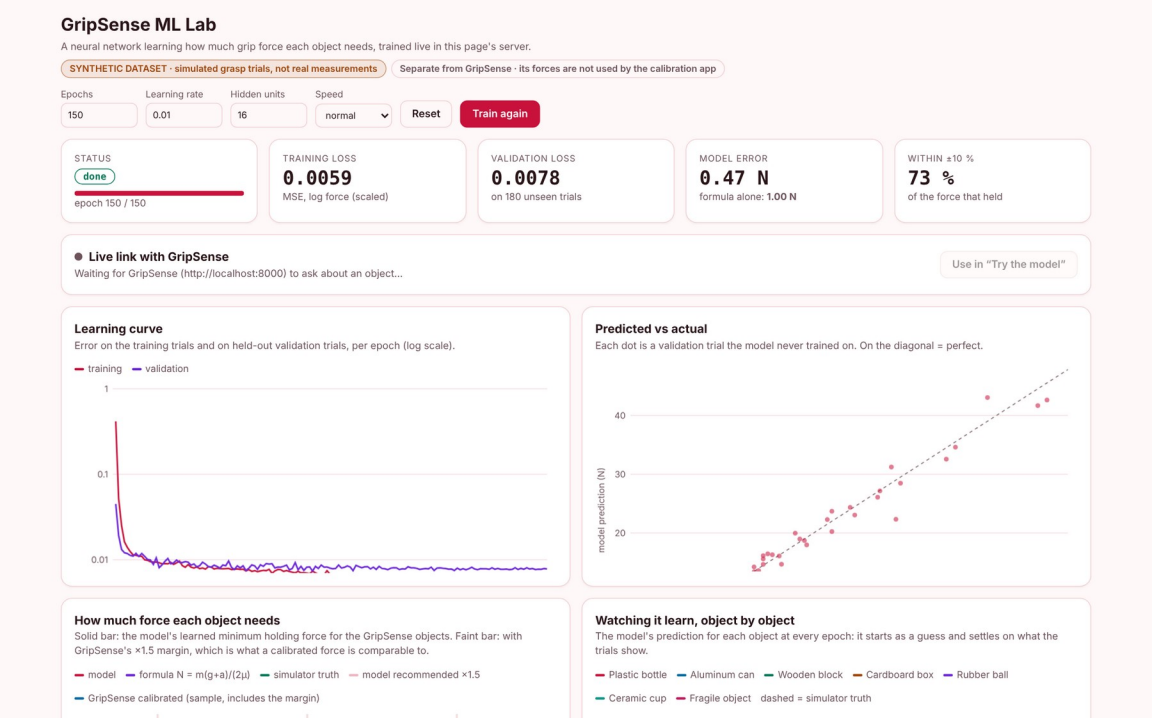


Figure 3.8: ML Lab Dashboard

Chapter 4: Results and Discussion

The results in this chapter come from the software running with simulated telemetry and from the automated tests; the system has not yet been run against the physical gripper.

4.1 Functional Results

- Selecting a calibrated object loads its force from the database without an LLM call; changing any property selects a different configuration and asks the LLM again; restoring the exact properties finds the saved calibration again.
- An untested or unconfirmed force is never saved as calibrated, and a failed calibration never becomes active.
- In a demonstrated session the fragile object slipped at 1.0 N; the assistant suggested 3.5 N, which held, and after confirmation the stepper reached “Saved and reused” and the gallery showed the object as calibrated.
- The 3D view and the charts follow the telemetry; in live mode with no connection, readings show as unavailable and no simulated values are substituted.

4.2 ML Model Results

On the 180 validation trials the trained model’s mean absolute error is 0.47 N, against 1.00 N for the friction formula alone, and 73% of its predictions are within ±10% of the force that held (Table VIII). The learning curve flattens after about 20 epochs (Figure 4.1); the remaining error is mostly the noise and the slider step built into the data.

Table VIII: ML Results on Unseen Trials

Measure	ML model	Friction formula
Mean absolute error	0.47 N	1.00 N
Within ±10% of the force that held	73%	-
Validation trials	180	180

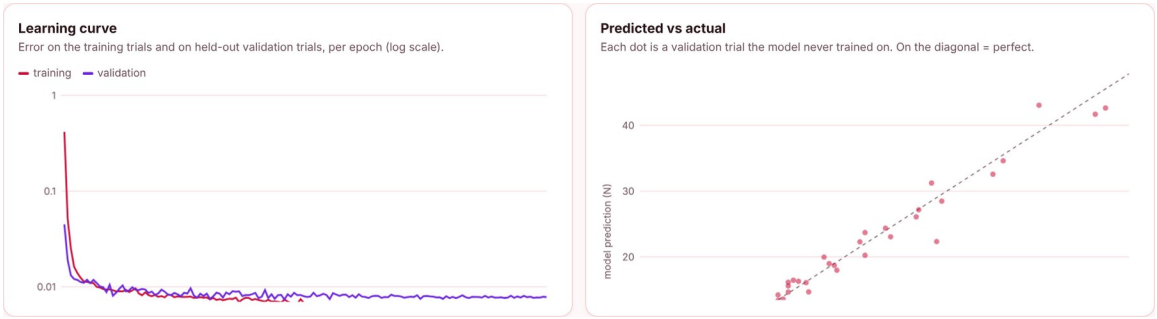


Figure 4.1: ML Learning Curve and Predicted versus Actual Force

Table IX gives the learned force for the GripSense objects. The model’s minimum is close to the simulator’s hidden truth and above the formula for slippery materials, and the recommended value

(×1.5) is comparable with the forces calibrated in the sample data, which include the margin. The model learned the material effects well (Figure 4.3) and the narrow-object effect only partly, because that effect is small compared with the noise.

Table IX: Learned Force per Object (N)

Object	Model min	Recommended ×1.5	Formula	Simulator truth	Sample calibrated
Plastic bottle	5.98	8.96	5.17	5.62	9.50
Aluminium can	7.73	11.59	6.41	7.29	12.00
Wooden block	2.80	4.20	2.62	2.50	5.50
Cardboard box	3.23	4.85	2.95	2.95	4.00
Rubber ball	1.31	1.96	0.98	0.88	1.50
Ceramic cup	4.29	6.43	3.54	4.17	4.50
Fragile object	1.86	2.79	1.57	1.97	-

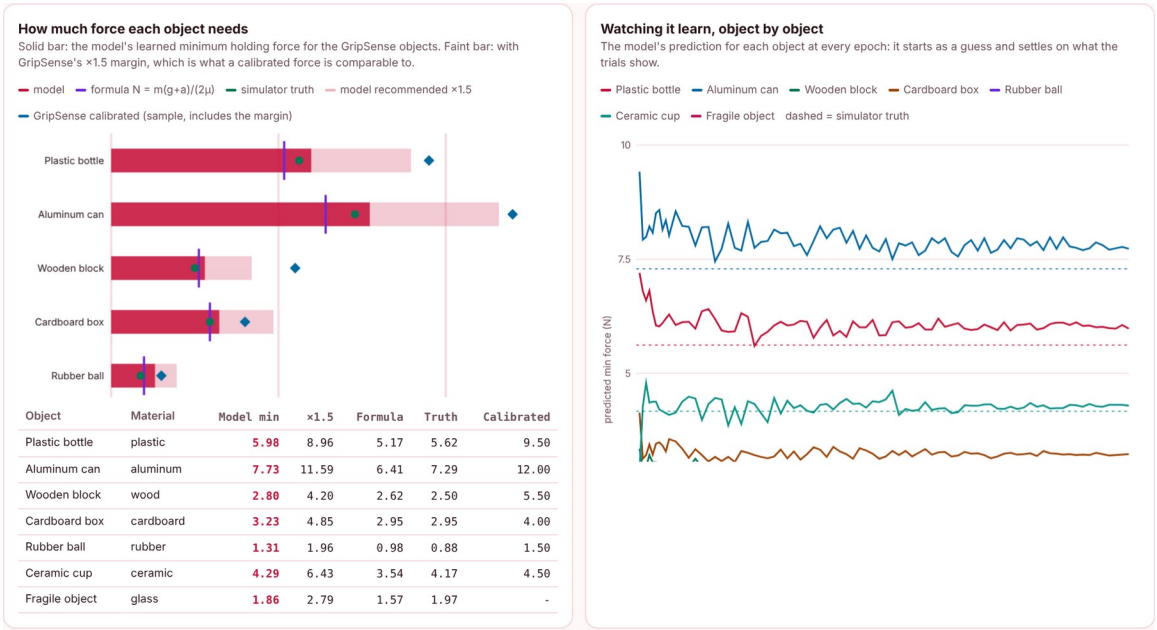


Figure 4.2: Learned Force for Each Object

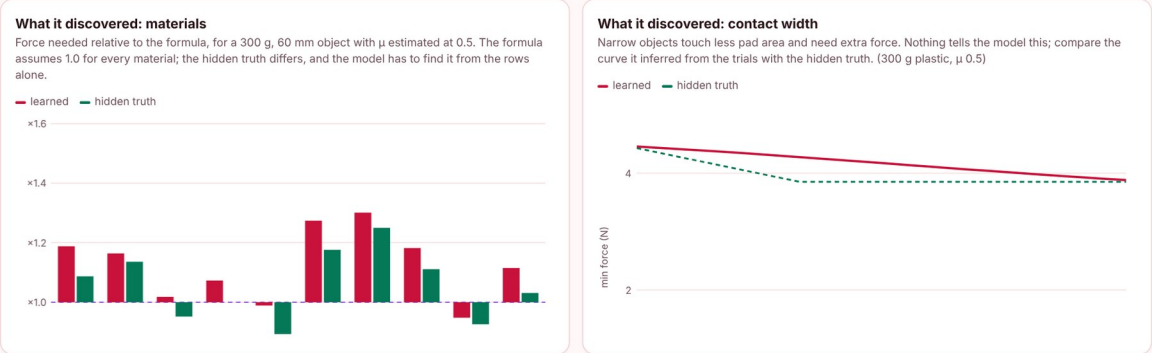


Figure 4.3: Material Effects Learned by the Model

4.3 Testing

Table X: Automated Tests

Suite	Tests	Covers
Backend (pytest)	135	Reuse without LLM call, safety rules, calibration rules, database emulation, failure handling, ML link
Frontend (Vitest)	141	Force display, slider, safety cases, 3D kinematics, telemetry, history, palette
ML Lab (pytest)	3	Dataset, model beats the formula, server
Total	279	All passing

The pages were also checked in a browser in both themes and on a phone-sized screen, with no page errors.

4.4 Implementation Status

Table XI: Implementation Status

Function	Status
Backend, safety model, calibration workflow	Complete
Web console, 3D view, telemetry, history	Complete
Database layer, sample data, validator	Complete; tested on an in-memory emulation
ML Lab and its link to the console	Complete; trained on synthetic data
Offline demonstration mode	Complete
Connection to the project's Firebase database	Configured; live test pending

Function	Status
Test with the physical gripper and real telemetry	Pending
Training the model on real calibrations	Pending

4.5 Comparison with Existing Approaches

Table XII: Comparison with Existing Approaches

Approach	Force decided by	What the present project adds
Fixed force	A constant	A force per object configuration
Friction formula [1]	$m(g + a)/(n \cdot \mu)$ with a margin	Formula used as a check, value confirmed by test
Learned grasping [5], [6]	A network trained on many trials	Small model on object properties, compared with the formula
LLM suggestion [8], [9]	Model output	Used once, validated, then replaced by the confirmed value
This project	Saved calibration first, then LLM, then test and confirm	Reuse without LLM calls and a full audit trail

4.6 Discussion

The workflow achieves its main aim: a force is estimated once, confirmed by a person on the gripper, and reused automatically. The ML experiment shows that a small network can learn effects the friction formula misses and halve its error, but the result holds only for the synthetic data it was trained on; real calibrations are needed before its predictions can be trusted. The remaining work is the integration with the physical gripper: confirming the force limits with the hardware team, connecting the real telemetry, and running the calibration on real objects.

Chapter 5: Conclusion and Future Scope

5.1 Conclusion

This mini project developed GripSense AI, the software for a force-modulating parallel-jaw gripper. It resolves a grip force from a saved calibration or, when none exists, from one LLM estimate; checks every force against a friction model and the limits; guides the operator through testing, slider adjustment and explicit confirmation; and saves and reuses confirmed forces per object configuration in a Firebase Realtime Database. A web console with a 3D work-cell, live telemetry, history and replay makes the process visible, and an ML Lab trains a neural network that predicts the force with 0.47 N error on unseen synthetic trials, against 1.00 N for the formula. The software is covered by 279 automated tests and runs offline for demonstration; tests with the physical gripper and real data remain.

5.2 Future Scope

- Connect the physical gripper and its fingertip force sensors to the telemetry node and calibrate real objects.
- Run the system against the project's Firebase database with secured rules and user sign-in.
- Retrain the ML model on real confirmed calibrations and use it as an initial estimate beside the LLM.
- Send the target force to the gripper through an agreed command channel with acknowledgement.
- Use the measured force for closed-loop control of the grip.

References

[1] G. J. Monkman, S. Hesse, R. Steinmann, and H. Schunk, Robot Grippers. Weinheim, Germany: Wiley-VCH, 2007.

[2] S. Ramírez, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com

[3] Google, “Firebase Realtime Database documentation.” [Online]. Available: https://firebase.google.com/docs/database

[4] Meta Platforms, “React documentation.” [Online]. Available: https://react.dev

[5] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, “Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection,” Int. J. Robot. Res., vol. 37, no. 4–5, pp. 421–436, 2018.

[6] J. Mahler et al., “Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics,” in Proc. Robotics: Science and Systems (RSS), 2017.

[7] R. Calandra et al., “More than a feeling: Learning to grasp and regrasp using vision and touch,” IEEE Robot. Autom. Lett., vol. 3, no. 4, pp. 3300–3307, 2018.

[8] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” in Proc. Conf. Robot Learning (CoRL), 2022.

[9] T. Brown et al., “Language models are few-shot learners,” in Advances in Neural Information Processing Systems 33, 2020.

[10] three.js authors, “three.js documentation,” and Poimandres, “React Three Fiber documentation.” [Online]. Available: https://threejs.org/docs, https://r3f.docs.pmnd.rs

[11] Pydantic, “Pydantic documentation.” [Online]. Available: https://docs.pydantic.dev

[12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[13] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.